SAPI. Advertising your server-side technology stack is just asking for trouble, and we'll see how we can modify that behavior in a moment. First, let's break down the status line and headers in more detail.

Response Status

Much like HTTP request methods, HTTP status codes are governed by a set of RFCs and maintained in an official registry by the IANA. The full registry of status codes can be found at `https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml`. The codes themselves are three-digit numbers where the first digit indicates the class of response.

Table 11-3. *HTML status code groups*

`1xx`	Informational (request received but not processed)
`2xx`	Success (request was understood and accepted, usually with a response)
`3xx`	Redirection (details in the `Location:` header)
`4xx`	Client error (you did something wrong)
`5xx`	Server error (we did something wrong)

You won't need to worry about a lot of these codes, because PHP is only going to get to work once the request is fully received and ready for processing. Anything involving, say, switching protocols (`101 Switching Protocols`, usually to tell a client to move from HTTP to HTTPS) or proxy issues (`502 Bad Gateway`) is going to be dealt with by the web server before control passes to your program. As a PHP developer, you'll mostly be dealing with setting `4xx` and `5xx` to communicate error states, perhaps `3xx` if redirection is handled in your application logic (rather than server config). By default, PHP will set the status code to `200` when running in a web server SAPI, and the function `http_response_code()` can be used to get the current status or set a new one:

```
// index.php
$oldCode = http_response_code();
http_response_code(403);

print "HTTP Status Code was {$oldCode}\n";
print "But I don't like you so you are forbidden.\n";
```